BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 016 · Noughts & Crosses · Part 4 — Complete and test

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What does MakeMove() return after a successful move?

Q2. Why must an invalid move not switch player?

Q3. What row and column does move 8 map to?

Q4. How many winning lines exist on a 3 × 3 Noughts & Crosses board?

Do Now Recall previous learning | **I Do** Read new knowledge | **We Do** Practise with guidance | **You Do** Work independently | **Review & Improve** Check understanding

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

Recall the term — Boolean

Recall the term — Condition

Recall the term — State

Recall the term — Game loop

Recall the term — Test case

Recall the term — Trace

Recall the term — Debug

New learning

HasWon() checks 8 lines — CODE:
3 rows
3 columns
2 diagonals
————————

8 winning lines — Each winning line needs three comparisons joined with && . The possible lines are joined with || .

BoardFull() looks for any free square — CODE:
if (board[row, col] != "X" &&
board[row, col] != "O")
{
return false;
}

return true; — If even one square is free, the board is not full.

Finish Main() — CODE:
valid move
→ check win
→ check draw
→ otherwise switch player — Part 3 already handled valid moves and switching. Today we add the stopping conditions.

Can two pupils actually play it? — After completing the code, play against a partner and compare expected with actual results. Use planned board positions to test all eight winning lines.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Worked example

Build BoardFull()

1. Use nested loops — CODE:
for (int row = 0; row < board.GetLength(0); row++)
{
for (int column = 0; column < board.GetLength(1); column++)
{
}
}

2. One free square means not full — CODE:
if (board[row, column] != "X" &&
board[row, column] != "O")
{
return false;
} — The function can stop immediately when it finds a free square.

3. No free square was found — CODE:
return true; — This line is reached only after every position has been checked.

4. Check draw after checking win — CODE:
if (HasWon(currentPlayer))
{
// win
}
else if (BoardFull())
{
// draw
}

We Do · Practise with guidance

Complete one diagonal

Player X wins on the main diagonal when all three diagonal positions contain X.

Complete the condition

```
if (board[0,0] == ? &&
    board[1,1] == ? &&
    board[2,2] == ?)
{
    return true;
}
```

```
if (board[0,0] == player &&
    board[1,1] == player &&
    board[2,2] == player)
{
    return true;
}
```

Spot a free square

A square is free when it contains neither X nor O.

Build the Boolean test

Put the actions in order

A valid move has just been made. What should happen next?

Complete the sequence

Choose the test

```
X O X
X O O
O X X
```

No winning line exists and every square is occupied.

Predict the functions


```
}
```

Final test: play more than once and deliberately test invalid moves, each type of win and a draw.

Exit Ticket · Model answer

True. After a valid move, check for a win before checking whether the board is full and the game is a draw.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

```
return "O";
}
else
{
return "X";
}
}

public static bool HasWon(string player)
{
return
(board[0,0] == player && board[0,1] == player && board[0,2] == player) ||
(board[1,0] == player && board[1,1] == player && board[1,2] == player) ||
(board[2,0] == player && board[2,1] == player && board[2,2] == player) ||

(board[0,0] == player && board[1,0] == player && board[2,0] == player) ||
(board[0,1] == player && board[1,1] == player && board[2,1] == player) ||
(board[0,2] == player && board[1,2] == player && board[2,2] == player) ||

(board[0,0] == player && board[1,1] == player && board[2,2] == player) ||
(board[0,2] == player && board[1,1] == player && board[2,0] == player);
}

public static bool BoardFull()
{
for (int row = 0; row < board.GetLength(0); row++)
{
for (int column = 0; column < board.GetLength(1); column++)
{
if (board[row, column] != "X" &&
board[row, column] != "O")
{
return false;
}
}
}

return true;
}
```


You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Finish all 8 winning lines
Continue your Lesson 015 program.
Complete HasWon(string player) so it detects all 3 rows, all 3 columns and both diagonals.

Question 2

Detect a draw
Create BoardFull() .
Return false as soon as a square contains neither X nor O. Return true only if every square is occupied.

Question 3

Complete the game loop
Draw the board, ask the current player for a move, call MakeMove() , then check for a win or draw.
Switch player only after a valid move that did not end the game.

Question 4

Partner challenge: play and test
Pair up: one pupil plays X and the other O. Run your own C# program, make moves 1–9 and swap who goes first for a second complete game.
Evidence: for each targeted test, record the input or board state, expected result, actual result and pass/fail. Fix and retest any failure. Test all eight winning lines using prepared moves or board states; ordinary matches alone may not cover them.

If your online C# runner does not support repeated keyboard input, run the program in a local C# console or use planned input sequences for testing.

Exit Ticket · Answer before checking

After a valid move, the program should check HasWon() before BoardFull() .


```
Console.Write(" | ");
}
}

Console.WriteLine();

if (row < board.GetLength(0) - 1)
{
Console.WriteLine("--+---+--");
}
}

Console.WriteLine();
}

public static bool MakeMove(int move, string player)
{
if (move < 1 || move > 9)
{
return false;
}

int row = (move - 1) / 3;
int column = (move - 1) % 3;

if (board[row, column] == "X" ||
board[row, column] == "O")
{
return false;
}

board[row, column] = player;
return true;
}

public static string SwitchPlayer(string player)
{
if (player == "X")
{
```

```
if (MakeMove(move, currentPlayer))
{
if (HasWon(currentPlayer))
{
DrawBoard();
Console.WriteLine("Player " + currentPlayer + " wins!");
gameOver = true;
}
else if (BoardFull())
{
DrawBoard();
Console.WriteLine("Draw!");
gameOver = true;
}
else
{
currentPlayer = SwitchPlayer(currentPlayer);
}
}
else
{
Console.WriteLine("Invalid move - try again.");
}
}
}

public static void DrawBoard()
{
Console.WriteLine();

for (int row = 0; row < board.GetLength(0); row++)
{
for (int column = 0; column < board.GetLength(1); column++)
{
Console.Write(board[row, column]);

if (column < board.GetLength(1) - 1)
{
```


Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. true.

Q2. The same player needs another attempt.

Q3. [2,1].

Q4. 8: 3 rows, 3 columns and 2 diagonals.

You Do pages · Independent questions

Q1. HasWon()
CODE:

```
public static bool HasWon(string player)
{
return
(board[0,0] == player && board[0,1] == player && board[0,2] == player) ||
(board[1,0] == player && board[1,1] == player && board[1,2] == player) ||
(board[2,0] == player && board[2,1] == player && board[2,2] == player) ||

(board[0,0] == player && board[1,0] == player && board[2,0] == player) ||
(board[0,1] == player && board[1,1] == player && board[2,1] == player) ||
(board[0,2] == player && board[1,2] == player && board[2,2] == player) ||

(board[0,0] == player && board[1,1] == player && board[2,2] == player) ||
(board[0,2] == player && board[1,1] == player && board[2,0] == player);
}
```

Check: 3 rows + 3 columns + 2 diagonals = 8 possible winning lines.

Q2. BoardFull()
CODE:

```
public static bool BoardFull()
{
for (int row = 0; row < board.GetLength(0); row++)
{
for (int column = 0; column < board.GetLength(1); column++)
{
```

```
if (board[row, column] != "X" &&
board[row, column] != "O")
{
return false;
}
}
}

return true;
}
```

Logic: one free square is enough to return false . Only return true after every square has been checked.

Q3. Main() game loop

CODE:

```
public static void Main()
{
string currentPlayer = "X";
bool gameOver = false;

while (gameOver == false)
{
DrawBoard();

Console.Write("Player " + currentPlayer + ", choose 1-9: ");
int move = int.Parse(Console.ReadLine());

if (MakeMove(move, currentPlayer))
{
if (HasWon(currentPlayer))
{
DrawBoard();
Console.WriteLine("Player " + currentPlayer + " wins!");
gameOver = true;
}
else if (BoardFull())
{
DrawBoard();
Console.WriteLine("Draw!");
```



```
gameOver = true;
}
else
{
currentPlayer = SwitchPlayer(currentPlayer);
}
}
else
{
Console.WriteLine("Invalid move - try again.");
}
}
}
```

Order matters: check win first, then draw, then switch player.

Q4. Complete playable version

CODE:

```
using System;

public class Program
{
public static string[,] board =
{
{ "1", "2", "3" },
{ "4", "5", "6" },
{ "7", "8", "9" }
};

public static void Main()
{
string currentPlayer = "X";
bool gameOver = false;

while (gameOver == false)
{
DrawBoard();

Console.Write("Player " + currentPlayer + ", choose 1-9: ");
int move = int.Parse(Console.ReadLine());
```